Week 10 · LangGraph Orchestration
The Complete Guide

A hands-on walkthrough from “what is a state graph” to “production multi-agent system with human-in-the-loop.” Read, run, build.

Why you should care — the 2-minute hook

Most AI demos are beautiful in a notebook and terrifying in production. The LLM is smart. The glue around it is not. You wire up a prompt, a tool, a retry loop, and by the time a real user hits it — session state is gone, an agent’s mid-call retry hits a duplicate payment, your observability is print() statements, and you cannot explain to your CTO why the cost tripled overnight.

This guide teaches the glue.

By the time you finish, you will have built a 3-agent customer-support system that:

- Classifies incoming tickets into three domains and hands off to specialists
- Persists its state to disk and survives a process restart
- Pauses for human approval before sending anything risky, then resumes exactly where it stopped
- Traces every node — inputs, outputs, tokens, latency, state diff — into an observability UI
- Composes a LangGraph workflow with the Claude Agent SDK subagent pattern, the current Anthropic stack

You will write roughly 200 lines of code across five progressive exercises. The final system is the same shape as the multi-agent workflows that, per LangChain’s v1.0 announcement (October 2025), power production agents at Uber, JP Morgan, BlackRock, Cisco, and others. The LangChain/LangGraph ecosystem sees 90M monthly downloads. Gartner projects that 40% of enterprise applications will embed task-specific AI agents by end of 2026, up from less than 5% a year ago.

If you build anything that looks like an agent, you will at some point need the mental model in this guide. You can learn it the easy way — two hours, one focused read — or later, the hard way, at 2am on a Tuesday.

Let’s make it the easy way.

What you will have at the end

- A cloned repo with 5 working notebooks and 5 reference solutions
- A live LangSmith project showing your agents’ traces
- A mental model for when to reach for LangGraph and when to reach for the Claude Agent SDK — and when to use both
- A concrete path from today’s exercise to a production multi-agent system in your own stack

---

Prerequisites — get these sorted before the hands-on

You need:

1. Python 3.11+. Check with `python3 --version`.
2. uv for fast package installs: `curl -LsSf https://astral.sh/uv/install.sh | sh`
3. A free Google AI Studio API key for Gemini. Get one at aistudio.google.com/apikey. **Important:** Google cut free-tier quotas sharply in December 2025 and stopped publishing static per-model numbers — the official rate-limits page now directs you to your AI Studio dashboard for your account's current caps. Limits vary by tier and account status and are enforced per project — check your active limits in AI Studio before class. For this workshop, **use Flash-Lite** if it's available on your account — it generally ships with the most permissive free-tier caps and is more than capable for the exercises.
4. Optional. An Anthropic API key for Exercise 5. Sign up at console.anthropic.com.
5. Optional. A free LangSmith account for Exercise 4. smith.langchain.com.

Set up the environment

```
git clone <this-repo>
cd w10-langgraph-hands-on

uv venv && source .venv/bin/activate
uv pip install -r requirements.txt

cp .env.example .env
# Edit .env and put your GOOGLE_API_KEY in it.

python verify_setup.py
# Expected output: all imports green, GOOGLE_API_KEY found, Gemini smoke test
passes
```

If `verify_setup.py` returns anything red, fix before going further. Most common failure: `GOOGLE_API_KEY` not picked up because `.env` wasn't loaded in the shell. Either `source .env` first or `export GOOGLE_API_KEY=...` manually.

PART I — The mental model

Why state graphs, not chains

Most first-attempt agents look like a chain:

```
prompt → LLM → tool → LLM → response
```

This works until it doesn't. The moment you need to:

- **Retry** a step without re-running the whole chain
- **Branch** based on what the LLM said
- **Pause** to ask a human something
- **Resume** after a restart
- **Loop** until some condition is met

…a chain falls over. You start patching with flags, retry counters, checkpointing hacks. You're reinventing a state machine — poorly.

LangGraph gives you the state machine as a primitive. Three pieces:

1. **State** — a `TypedDict` describing what your agent knows at any moment
2. **Nodes** — pure functions that take state and return a state delta
3. **Edges** — the rules that determine which node runs next (linear, conditional, looping)

That's the whole surface. Everything else — supervisors, swarms, checkpointers, HITL interrupts — composes these three primitives. You learn them once, and every pattern downstream just clicks.

Why not CrewAI or a simpler framework?

CrewAI and role-based wrappers are great for prototyping. You get a supervisor, workers, and a sequential or hierarchical orchestration with 20 lines. But they hide the state. The moment you need fine-grained control — "retry node X if it returns empty," "pause if the classifier confidence is below 0.7," "branch based on a field in the payload" — you're back to modifying internal machinery.

LangGraph is verbose up front and clean under pressure. Production teams tend to start on the high-level frameworks and migrate to LangGraph when complexity demands it. This guide skips the migration.

The multi-agent pattern in 2026

Anthropic's 2026 Agentic Coding Trends Report describes the pattern that is replacing single-agent workflows in production: **multi-agent architectures where an orchestrator coordinates specialized agents working in parallel**, each with dedicated context, then synthesizes results into integrated output. A human stamps approval on the risky step. State persists across every handoff.

Every exercise in this guide builds one piece of that pattern.

PART II — Five hands-on exercises

Each exercise takes 20–30 minutes. Run them in order; they build on each other.

Exercise 1 — Hello StateGraph

What you're building. A two-node graph that takes a support ticket body, classifies it into `TECHNICAL`, `BILLING`, or `GENERAL`, and returns a templated acknowledgment.

The state. The ticket, the category once classified, and the response once written.

```
from typing import Literal, TypedDict

Category = Literal["TECHNICAL", "BILLING", "GENERAL"]

class SupportState(TypedDict):
    ticket_body: str
    category: Category | None
    response: str | None
```

The nodes. Each node is a plain function. It receives state, does work, returns a dict containing only the fields it changed.

```
from langchain_google_genai import ChatGoogleGenerativeAI
from dotenv import load_dotenv
load_dotenv()

llm = ChatGoogleGenerativeAI(model="gemini-2.5-flash-lite", temperature=0)

def classify(state: SupportState) -> SupportState:
    prompt = (
        "Classify this support ticket into ONE of: TECHNICAL, BILLING,
GENERAL.\n"
        "Reply with ONLY the category word.\n\n"
        f"Ticket: {state['ticket_body']}"
    )
    result = llm.invoke(prompt).content.strip().upper()
    if result not in ("TECHNICAL", "BILLING", "GENERAL"):
        result = "GENERAL"
    return {"category": result}

def respond(state: SupportState) -> SupportState:
    template = f"Thanks — your {state['category']} ticket has been received.
Ref: T-XXXXX."
    return {"response": template}
```

The graph. Declare nodes and edges, compile, invoke.

```
from langgraph.graph import StateGraph, START, END

graph = StateGraph(SupportState)
graph.add_node("classify", classify)
graph.add_node("respond", respond)
graph.add_edge(START, "classify")
graph.add_edge("classify", "respond")
```

```
graph.add_edge("respond", END)

app = graph.compile()

# Visualize — LangGraph will draw it for you
print(app.get_graph().draw_ascii())

# Run it
result = app.invoke({
    "ticket_body": "API is returning 500 errors since this morning.",
    "category": None,
    "response": None,
})
print(result)
```

> **Stop and run it.** Open `notebooks/01_hello_graph.py` in VS Code, Cursor, or PyCharm. The `# %%` comments make it open as a native notebook. Run each cell. You should see the ASCII diagram of your graph and the classification output.

What clicked. State is a typed dictionary. Nodes return only the fields they changed — LangGraph merges the deltas into the full state. Edges are the glue. That's the whole primitive; everything else is an elaboration.

Common gotcha. Nodes that return `None` or return a dict without the field they're supposed to set will cause downstream nodes to see `None` and behave oddly. Always return a dict with the updated field explicitly.

Challenge. Add a `priority` field derived from keyword match (`down`, `error`, `500` → HIGH) before `classify`. Print the ASCII graph again to see your architecture grow.

Exercise 2 — The tool-calling supervisor

What you're building. The classify node from Exercise 1 gets replaced by a *supervisor agent* that routes the ticket to one of three *specialist tools*. This is the multi-agent primitive.

Why this matters. LangChain's multi-agent guide recommends building the supervisor as a tool-calling LLM directly, rather than pulling in the dedicated `langgraph-supervisor` library. The reason is context engineering: when the supervisor is just an LLM calling tools you defined, you control exactly what it sees and how it decides. The library wrapper hides that.

Specialists as tools.

```
from langchain_core.tools import tool

@tool
def technical_specialist(ticket: str) -> str:
    """Handle TECHNICAL support questions (APIs, devices, dashboards, syncs,
errors)."""
    return llm.invoke(
        f"You are a senior technical support engineer. Answer crisply:\n\
n{ticket}"
    ).content

@tool
def billing_specialist(ticket: str) -> str:
    """Handle BILLING questions (charges, invoices, plans, refunds)."""
```

```
    return llm.invoke(
        f"You are a billing specialist. Be precise and reassuring:\n\
n{ticket}"
    ).content

@tool
def general_specialist(ticket: str) -> str:
    """Handle GENERAL or pre-sales questions (SLA, references, demos,
integrations)."""
    return llm.invoke(
        f"You are a friendly pre-sales specialist. Be helpful and concise:\n\
n{ticket}"
    ).content

SPECIALISTS = [technical_specialist, billing_specialist, general_specialist]
```

The supervisor agent. `create_react_agent` from `langgraph.prebuilt` gives you a tool-calling ReAct loop for free.

```
from langgraph.prebuilt import create_react_agent

supervisor_prompt = (
    "You are the support supervisor. Read the ticket, pick exactly ONE
specialist tool, "
    "call it with the original ticket text, then stop. Do not answer the
ticket yourself."
)

supervisor = create_react_agent(
    model=llm,
    tools=SPECIALISTS,
    prompt=supervisor_prompt,
)
```

Wrap it in a StateGraph.

```
import operator
from typing import Annotated, TypedDict

class SupervisorState(TypedDict):
    ticket_body: str
    messages: Annotated[list, operator.add]
    final_response: str | None

def run_supervisor(state: SupervisorState) -> SupervisorState:
    result = supervisor.invoke({"messages": [("user", state["ticket_body"])]})
    return {"final_response": result["messages"][-1].content, "messages":
result["messages"]}

graph = StateGraph(SupervisorState)
graph.add_node("supervisor", run_supervisor)
graph.add_edge(START, "supervisor")
graph.add_edge("supervisor", END)
app = graph.compile()
```

```
result = app.invoke({"ticket_body": "I was charged twice last month.",
"messages": [], "final_response": None})
print(result["final_response"])
```

> **Stop and run it.** Open `notebooks/02_supervisor.py`. Try the three sample tickets at the bottom — each should route to a different specialist.

What clicked. The supervisor is not a special kind of agent. It's an LLM with tools, where the tools happen to be other specialist agents. The decorator `@tool` wraps a Python function so the LLM can call it via structured output. This is the 2026 pattern: every "agent" is ultimately an LLM-plus-tools, and "multi-agent" is just nested tool-calling.

Forward compatibility note. `create_react_agent` was deprecated when LangGraph 1.0 shipped (October 2025) in favor of `create_agent`, which ships in LangChain 1.0's `langchain.agents` package — not LangGraph itself. The prebuilt still works in current versions, but the migration path is published. For production code, read the agents guide and move to `create_agent`.

Common gotcha. The supervisor will sometimes answer the ticket itself rather than routing. Two fixes: (1) make the prompt more strict ("do not answer the ticket yourself"), (2) remove any generic `respond_to_user` tool that competes with your specialists.

Challenge. Add a fourth specialist `escalation_specialist` that catches cancellation threats or abusive language. Update the supervisor prompt to route to it when appropriate. See `solutions/02_supervisor_solution.py` for a reference.

Exercise 3 — Checkpointing + human-in-the-loop

What you're building. A three-node graph — `draft → human_approval → send` — where the second node *pauses the graph* to wait for a human decision. The entire state is persisted to SQLite so it survives a process restart.

Why this matters. Production agents make decisions that have real-world consequences: sending email, charging a card, updating a record. You need a pause-point for humans on the risky step. You also need durability — your agent should resume from where it stopped after a restart, not lose its place.

LangGraph's official docs call this the **interrupt primitive**. It's a function you call from inside a node that pauses execution and surfaces a payload to the caller. The caller resumes with `Command(resume=...)`.

The state.

```
class TicketState(TypedDict):
    ticket_body: str
    draft_response: str | None
    approved: bool | None
    sent_response: str | None
```

The nodes.

```
from langgraph.types import interrupt, Command

def draft(state: TicketState) -> TicketState:
    prompt = f"Draft a short, polite 2-sentence reply:\n\
n{state['ticket_body']}"
    return {"draft_response": llm.invoke(prompt).content}
```

```
def human_approval(state: TicketState) -> TicketState:
    decision = interrupt({
        "action": "approve_reply",
        "draft": state["draft_response"],
        "question": "Approve sending? (approve / edit / reject)",
    })
    if isinstance(decision, dict):
        if decision.get("action") == "approve":
            return {"approved": True}
        if decision.get("action") == "edit":
            return {"draft_response": decision["text"], "approved": True}
    return {"approved": False}

def send(state: TicketState) -> TicketState:
    if not state.get("approved"):
        return {"sent_response": "[REJECTED]"}
    return {"sent_response": f"SENT: {state['draft_response']}"}
```

Compile with a SQLite checkpointer.

```
import sqlite3
from langgraph.checkpoint.sqlite import SqliteSaver

conn = sqlite3.connect("checkpoints.db", check_same_thread=False)
memory = SqliteSaver(conn)

g = StateGraph(TicketState)
g.add_node("draft", draft)
g.add_node("human_approval", human_approval)
g.add_node("send", send)
g.add_edge(START, "draft")
g.add_edge("draft", "human_approval")
g.add_edge("human_approval", "send")
g.add_edge("send", END)

app = g.compile(checkpointer=memory)
```

Run it, interrupt, resume.

```
thread_id = "ticket-demo-001"
config = {"configurable": {"thread_id": thread_id}}

# Step 1: run until interrupt fires
state = app.invoke({
    "ticket_body": "My dashboard is broken.",
    "draft_response": None, "approved": None, "sent_response": None,
}, config=config)

print("Paused. Draft:", state["draft_response"])

# Step 2: simulate a human approving
state = app.invoke(Command(resume={"action": "approve"}), config=config)
print("Final:", state["sent_response"])
```

> **Stop and run it.** Open `notebooks/03_checkpointing.py`. Then as an extra experiment: run only Step 1, kill the Python process, restart, and run only Step 2 with the same `thread_id`. The state resumes from SQLite.

What clicked. `SqliteSaver` writes state to disk after every node. `interrupt()` pauses execution and surfaces a payload. `Command(resume=...)` delivers the human's decision back into the paused node. `thread_id` is the identity of this particular run — LangGraph uses it to look up the correct checkpoint.

Bridge back to Exercise 2. The supervisor in Ex 2 is itself a state graph internally. `create_react_agent` accepts a `checkpointer` argument. Pass a `SqliteSaver` and the supervisor becomes durable too:

```
supervisor = create_react_agent(
    model=llm,
    tools=SPECIALISTS,
    prompt=supervisor_prompt,
    checkpointer=memory,
)
```

Now your supervisor can be interrupted mid-tool-call, resumed after restart, and gated with interrupt() before risky specialist calls. Ex 2 and Ex 3 compose.

Common gotcha. If `interrupt()` doesn't pause, you forgot to compile with `checkpointer=memory`. The interrupt primitive requires a checkpointer — without one, state can't be saved, so there's nowhere to resume from.

Challenge. Re-run with `{"action": "edit", "text": "Thanks — fix ETA 1 hour."}` instead of approve. Then inspect the full state history with `for c in memory.list(config): print(c)`.

Exercise 4 — Observability with LangSmith

What you're building. Enable LangSmith on the same graph and compare a weak prompt against a strong one by reading the traces side-by-side.

Why this matters. You cannot fix what you cannot see. You cannot cost-control what you cannot measure. LangSmith captures every node's inputs, outputs, latency, token count, and state diff for every run.

Enable it. Set two env vars *before* importing LangChain. Kernel already running? Restart it.

```
import os
os.environ["LANGSMITH_TRACING"] = "true"
os.environ["LANGSMITH_PROJECT"] = "w10-hands-on"
# LANGSMITH_API_KEY is auto-picked up from your shell/.env
```

Weak prompt vs. strong prompt. Build two versions of the classify node and compare.

```
WEAK = "Categorize this ticket: {ticket}"
STRONG = (
    "You are a ticket classifier. Return exactly one of: TECHNICAL, BILLING,
GENERAL. "
    "Return just the word, no punctuation.\n\n"
    "Examples:\n"
    "  'My API is down' -> TECHNICAL\n"
    "  'Where is my invoice?' -> BILLING\n"
    "  'Do you have a demo?' -> GENERAL\n\n"
```

```
    "Ticket: {ticket}"
)
```

Run the same ticket through both graphs and open smith.langchain.com → project `w10-hands-on`. You'll see both traces. Diff them. The strong prompt should have higher accuracy and fewer tokens per request.

> **Stop and run it.** `notebooks/04_langsmith.py`. Open LangSmith in another tab while it runs. Click the traces as they appear.

What clicked. Observability is not a nice-to-have. It's the thing that lets you go from "the agent misbehaved once" to "here's the exact state, prompt, model response, and cost that produced the misbehavior — let me fix it." Without it, you're guessing.

Common gotcha. Missing traces usually mean `LANGSMITH_TRACING=true` was set *after* you imported LangChain. Restart the kernel and set env vars first.

Challenge. Generate 20 labeled fake tickets with `data/fake_tickets.py`, run both prompts over them, and report accuracy. Bonus: push the 20 to a LangSmith dataset via the SDK and use LangSmith's built-in eval UI.

Exercise 5 — Hybrid with the Claude Agent SDK

What you're building. A LangGraph node that invokes the Claude Agent SDK to run a *subagent*, equipped with an *Agent Skill* for specialized knowledge. This is the current Anthropic stack.

Why this matters. LangGraph and the Claude Agent SDK are not competitors. They complement:

- **LangGraph** owns workflow: what runs when, with what state, with what durability guarantees
- **Claude Agent SDK** owns agent execution: how each agent runs, with which tools, how subagents isolate context

Every LangGraph node can be powered by a Claude Agent SDK call. And the same `AgentDefinition` shape deploys unchanged to Claude Managed Agents, which Anthropic launched in public beta on April 8, 2026 — no waitlist, no infra.

Three Anthropic primitives in one exercise.

1. **Claude Agent SDK** — the framework itself
2. **Agent Skills** — structured, versioned, shareable specialized knowledge. See Agent Skills overview.
3. **Subagents** — separate agent instances spawned via the Agent tool to do focused work. See Subagents in the SDK.

The code.

```
import asyncio, os
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

SDK_AVAILABLE = bool(os.getenv("ANTHROPIC_API_KEY"))

class HybridState(TypedDict):
    ticket: str
    diagnosis: str | None
    answer: str | None
```

```
async def diagnose_with_sdk(state: HybridState) -> HybridState:
    if not SDK_AVAILABLE:
        return {"diagnosis": "[SDK skipped — no ANTHROPIC_API_KEY]"}

    from claude_agent_sdk import AgentDefinition, ClaudeAgentOptions, query

    researcher = AgentDefinition(
        description="Research specialist — investigates before acting",
        prompt=(
            "You investigate technical issues by breaking them into
components. "
            "Return a numbered diagnosis in under 80 words."
        ),
        tools=["Read", "Grep"],
        # NOTE: AgentDefinition.model only accepts short names:
        #   "sonnet" | "opus" | "haiku" | "inherit"
        # Full IDs like "claude-sonnet-4-6" go on ClaudeAgentOptions.model
(the orchestrator).
        model="sonnet",
    )

    options = ClaudeAgentOptions(
        system_prompt="You are a senior support engineer.",
        agents={"researcher": researcher},
        # The "Agent" tool must be allowed — it's how Claude invokes
subagents.
        allowed_tools=["Agent"],
        # setting_sources enables Claude Code's filesystem config (CLAUDE.md,
skills, hooks).
        # Use "project" to load from ./.claude/ and "user" for ~/.claude/.
        setting_sources=["user", "project"],
        max_turns=5,
    )

    chunks = []
    async for msg in query(
        prompt=f"Use the researcher subagent to diagnose: {state['ticket']}",
        options=options,
    ):
        if hasattr(msg, "content"):
            chunks.append(str(msg.content))
    return {"diagnosis": "\n".join(chunks) if chunks else "[no diagnosis]"}

def draft_answer(state: HybridState) -> HybridState:
    return {"answer": f"Diagnosis:\n{state['diagnosis']}\n\nFollow-up within 1
hour."}

g = StateGraph(HybridState)
g.add_node("diagnose", diagnose_with_sdk)
g.add_node("draft", draft_answer)
g.add_edge(START, "diagnose")
g.add_edge("diagnose", "draft")
```

```
g.add_edge("draft", END)
app = g.compile()

async def main():
    r = await app.ainvoke({
        "ticket": "API 500s since 7am and dashboard shows no data.",
        "diagnosis": None, "answer": None,
    })
    print(r["diagnosis"])
    print(r["answer"])

asyncio.run(main())
```

> **Stop and run it.** `notebooks/05_hybrid_sdk.py`. If you don't have an `ANTHROPIC_API_KEY`, the fallback returns a placeholder — you can still read the graph shape.

What clicked. Subagents are separate agent instances that your main agent can spawn to handle focused subtasks. They **isolate context** — the main agent only sees the subagent's final output, not its full internal tool trace. This keeps the main context window lean and lets you run multiple subagents in parallel. Skills package specialized knowledge in a discoverable, shareable format. Managed Agents removes the Docker/load-balancer/session-store infra you'd otherwise have to build yourself.

Common gotcha. Running async code from a Jupyter cell can raise `RuntimeError: This event loop is already running`. If it fires, use `await main()` directly instead of `asyncio.run(main())` in Jupyter, or run the script from the terminal.

Production deployment note. The same `AgentDefinition` shape above runs on Claude Managed Agents. The quickstart is at platform.claude.com/docs/en/managed-agents/quickstart. One production tip from Anthropic's multi-agent guide: multi-agent systems use about 15× more tokens than single-turn chats — make sure the task has enough value to justify it. On their internal research eval, the multi-agent setup outperformed single-agent Claude Opus 4 by 90.2%.

Challenge. Replace the single researcher subagent with two parallel subagents: one for log inspection, one for metric correlation. Merge their outputs. See `solutions/05_hybrid_sdk_solution.py`.

PART III — Production gotchas

Five traps that eat training budget and shipping time. All five come from real production postmortems.

Gotcha 1 — The async event-loop trap

Running `async for msg in query(...)` from a Jupyter cell throws `RuntimeError: This event loop is already running`. Fix: use `await` directly in Jupyter, or run as a plain `.py` from the terminal. For production, wrap everything in `asyncio.run()` at the entry point only, never inside.

Gotcha 2 — Rate limits you didn't plan for

Google cut Gemini free-tier quotas hard in December 2025 and **stopped publishing static per-model numbers** on the rate-limits page — it now directs you to your AI Studio dashboard. Per Google's current docs, free-tier limits vary by tier and account status, are enforced per project, and should be checked in AI Studio before you run anything at scale. A class of 15 people each doing 4–8 calls per invocation (as Ex 2 does) can blow through either a per-minute or *per-day* cap quickly — check the dashboard first.

Fixes: `temperature=0` (no retries), pick Flash-Lite when it's available on your account, and have a fallback ready — a shared paid key, Groq's free tier, or OpenRouter. In production, use a paid tier and respect the per-minute quota.

Gotcha 3 — The "I changed state but it didn't save" trap

Nodes must **return a dict with the updated field**. Mutating `state["foo"] = "bar"` in place does nothing — LangGraph merges the returned dict into canonical state. If your change isn't persisting, check that your node returns it.

Gotcha 4 — Missing END edge → infinite loop

Forgetting `graph.add_edge("last_node", END)` makes the graph think `last_node` has no successor, which in some cases triggers a loop back to its own output. Always terminate every path explicitly.

Gotcha 5 — LangSmith silently not tracing

If you set `LANGSMITH_TRACING=true` *after* importing `langchain_*`, the hook never installs. Set env vars first, or restart your kernel.

Gotcha bonus — create_react_agent is on a deprecation track

LangGraph 1.0 (October 2025) deprecated `create_react_agent` in favor of `create_agent`, which ships in *LangChain 1.0's* `langchain.agents` package — not LangGraph itself. The prebuilt still works in current versions but for long-lived production code, migrate to `create_agent`. See the official agents guide.

PART IV — From here to production

You've built a working multi-agent system. What's the gap to production? Four things.

1. Durable infrastructure

SQLite works for demos and single-machine deployments. For production, swap for Postgres checkpointer or Redis. Same interface, different storage. The graph code doesn't change.

2. Cost governance

Multi-agent systems use about 15× more tokens than chats, per Anthropic's own measurement. You need per-task and per-tenant cost caps. LangSmith gives you the visibility; your agent harness needs to enforce hard caps before a runaway loop burns the budget.

3. Governance and PII

Every inbound and outbound message should pass through a compliance layer: PII detection, policy enforcement, audit logging. Design a `govern_input` / `govern_output` node you can slot in before every LLM call. This is non-negotiable for regulated domains (healthcare, finance, legal).

4. Evaluation

Write an eval dataset — start with 20 labeled examples, grow to 500. Run it on every prompt or model change. LangSmith has built-in eval tooling for this. Without evals, you're shipping vibes.

The one insight to take home

Multi-agent orchestration in 2026 is not about picking the smartest framework. It's about **state discipline**. Every production issue — duplicate sends, lost sessions, runaway costs, invisible failures — traces back to undisciplined state.

LangGraph makes state the first-class object. You declare it, you mutate it through nodes, you persist it through checkpointers, you inspect it through traces. Master that, and every pattern — supervisors, swarms, hybrids — is just a different arrangement of the same primitives.

Homework

Pick three of the four:

1. **Persistence upgrade** — swap `SqliteSaver` for `PostgresSaver` in Ex 3. Commit a working branch.
2. **Escalation specialist** — already done in the solution for Ex 2. Re-implement yourself and add a unit test that proves cancellation tickets route to it.
3. **Parallel fan-out** — in Ex 2, call all three specialists in parallel using the `Send` API. Let the supervisor pick the best reply.
4. **Graph visualization** — render a graph with `app.get_graph().draw_mermaid_png()` and add the PNG to your repo.

Stretch goal. Port Exercise 5 to Claude Managed Agents (public beta, no waitlist). Deploy, hit it from the CLI, report round-trip latency.

Glossary

Agent. An LLM with tools and a loop. In this guide: create_react_agent(model, tools, prompt).

Agent Skill. A structured, discoverable unit of specialized knowledge, discoverable from setting_sources=["user", "project"] in the Claude Agent SDK.

Checkpointer. A storage backend that persists graph state after every node. SqliteSaver, PostgresSaver, etc.

Claude Agent SDK. Anthropic's agent framework, renamed from the Claude Code SDK in September 2025 when its broader applicability beyond coding became clear. Best for agent execution inside a node.

Claude Managed Agents. Anthropic's hosted runtime for production agents, launched in public beta on April 8, 2026.

Conditional edge. An edge whose destination is decided at runtime based on state, added via graph.add_conditional_edges(...).

interrupt(). LangGraph primitive that pauses a node and surfaces a payload for human decision.

LangGraph. The state-machine orchestration library on top of LangChain.

LangSmith. LangChain's observability and eval platform.

Node. A function that takes state and returns a state delta.

Send API. LangGraph's mechanism for spawning parallel node invocations over a list of inputs.

State. A TypedDict describing everything your agent knows at any moment.

StateGraph. The class that wires nodes, edges, and state into an executable graph.

Subagent. A Claude Agent SDK primitive for spawning a focused, context-isolated agent from a parent agent.

Supervisor pattern. A supervisor LLM routes work to specialist tools. The current recommended shape.

TypedDict. Python's typed dictionary, used by LangGraph as the default state schema container.

Trustworthy references

Canonical, current sources. Prefer these over third-party blog regurgitations.

LangGraph itself

- LangGraph docs (official)
- LangChain + LangGraph 1.0 announcement (Oct 2025) — source of the 90M downloads figure and customer list
- Multi-agent guide
- Multi-agent collaboration tutorial
- Hierarchical agent teams tutorial
- Interrupts (human-in-the-loop)
- Agents reference (create_agent forward path)
- LangChain Academy — free LangGraph course

Anthropic

- Building Effective AI Agents
- How we built our multi-agent research system (90.2% uplift and 15× token cost)
- Writing effective tools for AI agents
- Claude Agent SDK overview
- Subagents in the Agent SDK
- Agent Skills overview
- Agent Skills in the SDK
- Claude Managed Agents overview
- Claude Managed Agents quickstart
- Claude Cookbook

Industry data

- Gartner — 40% of enterprise apps with task-specific AI agents by 2026
- Gartner Strategic Predictions 2026
- Anthropic 2026 Agentic Coding Trends Report
- Deloitte 2026 Tech Trends — Agentic AI strategy

Model tooling

- Google AI Studio (Gemini free keys)
- Gemini API rate limits (current, dashboard-driven)
- OpenRouter — GLM-5.1 access
- Anthropic console (Claude keys)
- LangSmith (observability)

Closing

Three hours of focused practice. Five concepts — state, supervisor, checkpointing, observability, hybrid. A working multi-agent system on your laptop and a mental model that scales from weekend projects to enterprise deployment.

If you take only one thing from this page, take this: **the production problem is state discipline, and LangGraph is the cheapest way to acquire it.**

Go build.